

Học Sâu (Deep Learning)

Chương 13: Dự báo Chuỗi thời gian (Timeseries Forecasting)

Đại học Đông Á

July 23, 2026

Nội dung chương

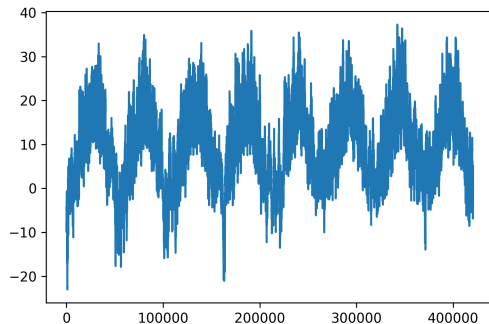
1. Giới thiệu Chuỗi thời gian và Bài toán Dự báo
2. Sự thất bại của các Mạng truyền thống
3. Mạng Nơ-ron Truyền hồi (Recurrent Neural Networks - RNN)
4. Kiến trúc LSTM - Long Short-Term Memory
5. Các Kỹ thuật Nâng cao cho RNN
6. Tổng kết

Chuỗi thời gian (Timeseries) là gì?

- Chuỗi thời gian là tập hợp các điểm dữ liệu được ghi nhận theo những khoảng thời gian đều đặn.
- Yếu tố "**Thứ tự**" (Temporal Order) mang tính quyết định. Quá khứ ảnh hưởng đến tương lai.
- Bài toán phổ biến nhất: **Dự báo (Forecasting)** - Dự đoán điều gì sẽ xảy ra tiếp theo.
- Ứng dụng: Dự báo tiêu thụ điện năng, dự báo doanh thu, dự báo thời tiết, hoặc giao dịch chứng khoán.

Khảo sát Dữ liệu Thời tiết Jena (Dài hạn)

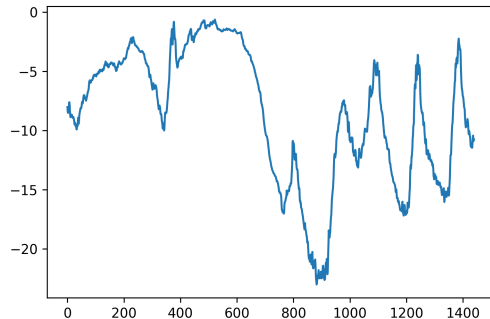
- Bộ dữ liệu từ Viện Max Planck, Đức. Ghi nhận 14 thông số thời tiết mỗi 10 phút.
- Trực quan hóa Biểu đồ Nhiệt độ trong khoảng thời gian nhiều năm.
- **Đặc điểm:** Có tính chu kỳ (Seasonality) rất rõ rệt theo các năm (Mùa đông lạnh, mùa hè nóng).



Hình 13.1: Nhiệt độ thay đổi qua nhiều năm

Khảo sát Dữ liệu Thời tiết Jena (Ngắn hạn)

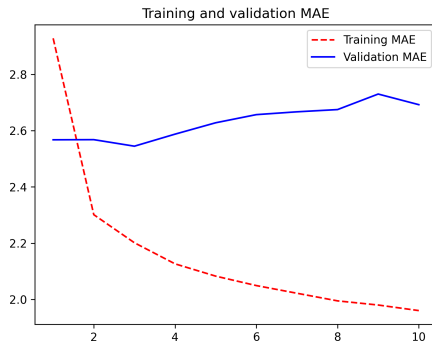
- Trực quan hóa chi tiết trong vòng 10 ngày (1440 điểm dữ liệu đầu tiên).
- Vẫn quan sát được tính chu kỳ theo ngày (Ngày ấm, đêm lạnh).
- Tuy nhiên, dữ liệu rất nhiễu và biến động mạnh do các yếu tố ngẫu nhiên.
- **Mục tiêu:** Dựa vào dữ liệu quá khứ, dự báo nhiệt độ của 24 giờ tiếp theo.



Hình 13.2: Nhiệt độ biến động trong vài ngày

Thử nghiệm với Baseline Densely-connected Model

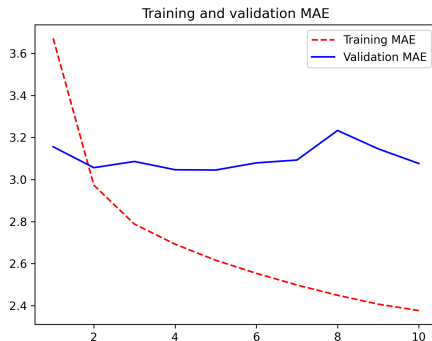
- Trải phẳng (Flatten) chuỗi thời gian và đưa qua các lớp Dense.
- **Kết quả:** Loss trên tập Validation (Đường liền) bị bão hòa nhanh chóng và Overfit.
- **Nguyên nhân:** Mạng Dense loại bỏ hoàn toàn khái niệm "Thời gian". Nó xem mọi điểm dữ liệu trong chuỗi là độc lập (IID).



Hình 13.3: Training Loss của Dense Model

Thử nghiệm với Mạng 1D Convolution

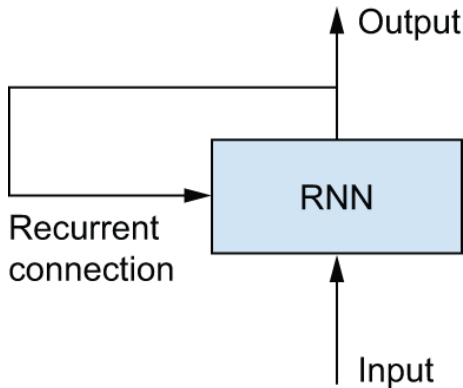
- Dùng Conv1D để trượt một "cửa sổ" qua chuỗi dữ liệu (Giống Conv2D trên ảnh).
- **Kết quả:** Tệ hơn cả Baseline cơ bản.
- **Lý do:** Conv1D giả định ranh giới Dịch chuyển Bất biến (Translation Invariance). Nghĩa là một khuôn mẫu ở thứ 2 có ý nghĩa giống hệt thứ 5. Điều này sai lầm trong Timeseries vì thời gian càng gần hiện tại càng mang thông tin quan trọng.



Hình 13.4: Training Loss của 1D Conv

Khái niệm Mạng Nơ-ron Truyền hồi (RNN)

- Trái ngược với mạng Feed-forward, RNN duy trì một **Trạng thái (State)** đóng vai trò như "Bộ nhớ".
- Nó duyệt qua từng bước thời gian t , tính toán Đầu ra (Output) tại t , và cập nhật lại Trạng thái để dùng cho $t + 1$.
- RNN lưu giữ thông tin về những gì nó đã thấy cho đến nay.



Hình 13.5: Vòng lặp trong mạng SimpleRNN

Vấn đề cốt lõi: Vanishing Gradients (Triệt tiêu Đạo hàm)

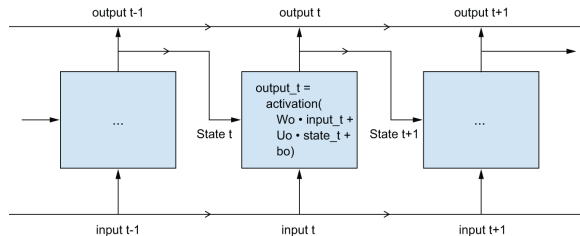
- **SimpleRNN** hoạt động tốt về mặt lý thuyết, nhưng thất bại trong thực tế.
- Khi độ dài chuỗi tăng lên, Đạo hàm (Gradient) từ tương lai phải nhân liên tục qua hàng trăm ma trận trọng số để quay ngược về quá khứ.
- Nếu trọng số < 1 , đạo hàm sẽ tiến về 0 cực nhanh (Triệt tiêu).
- Hậu quả: SimpleRNN bị "mất trí nhớ" ngắn hạn, quên mất các thông tin đầu chuỗi.

Giải pháp LSTM: Bổ sung Trạng thái Tế bào (Cell State)

- Được phát minh năm 1997, LSTM bổ sung một đường truyền dữ liệu song song gọi là **Cell State (C_t)**.
- Tưởng tượng C_t như một băng chuyền chạy dọc qua toàn bộ chuỗi. Thông tin có thể dễ dàng chảy xuyên suốt mà không bị thay đổi quá nhiều (Giải quyết hoàn toàn Vanishing Gradients).
- Các **Cổng (Gates)** sẽ quyết định thêm/bớt thông tin gì vào dải băng chuyền này.

LSTM (Phần 1): Trạng thái mang theo

- Tại thời điểm t , ta có Đầu vào X_t và Trạng thái ẩn từ quá khứ H_{t-1} .
- Thay vì chỉ đưa ra H_t như RNN thông thường, LSTM truyền thêm C_{t-1} vào ô tế bào hiện tại.



Hình 13.6: Giai đoạn khởi nhập của LSTM

LSTM (Phần 2): Forget Gate và Input Gate

Tính toán Tế bào trạng thái mới C_t :

- 1 **Forget Gate (f):** Quên bớt thông tin cũ không còn quan trọng. Hàm Sigmoid xuất giá trị 0 (Xóa) đến 1 (Giữ lại).

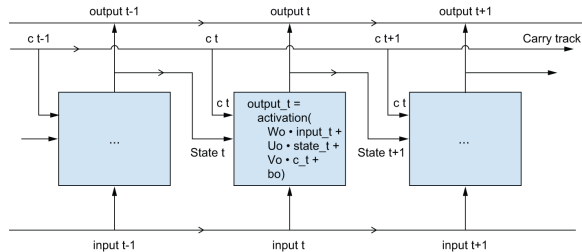
$$f = \sigma(W_f \cdot [X_t, H_{t-1}] + b_f)$$

- 2 **Input Gate (i và k):** Tạo thông tin mới để cập nhật vào băng chuyển.

$$i = \sigma(W_i \cdot [X_t, H_{t-1}] + b_i)$$

$$k = \tanh(W_k \cdot [X_t, H_{t-1}] + b_k)$$

- 3 $C_t = C_{t-1} * f + i * k$



Hình 13.7: Cập nhật Cell State C_t

LSTM (Phần 3): Output Gate

Cuối cùng, tính Trạng thái ẩn H_t (Cũng chính là Output):

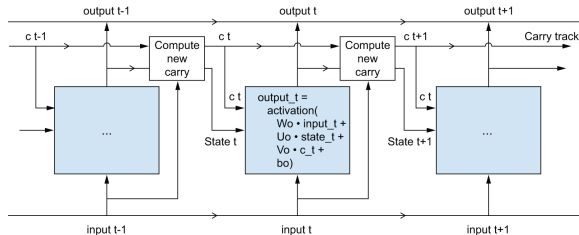
- **Output Gate (o):** Quyết định phần nào của Cell State sẽ được xuất ra.

$$o = \sigma(W_o \cdot [X_t, H_{t-1}] + b_o)$$

- Tính toán H_t :

$$H_t = o * \tanh(C_t)$$

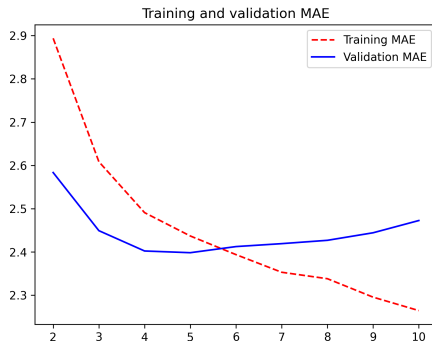
- Trạng thái ẩn H_t mang thông tin tinh lọc nhất để chuẩn bị bước sang thời điểm $t + 1$.



Hình 13.8: Xuất giá trị h_t

Hiệu suất Đột phá của LSTM

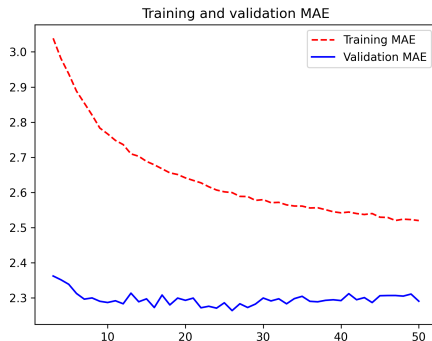
- Thay thế Baseline Dense bằng LSTM.
- **Kết quả:** Validation Loss thấp hơn hẳn, độ chính xác vượt qua Baseline suy luận thông thường (Common-sense baseline).
- Khả năng bắt được chuỗi giá trị và ghi nhớ cấu trúc thời tiết trong dài hạn đã phát huy tác dụng.



Hình 13.9: Biểu đồ Loss của LSTM

Kỹ thuật 1: Recurrent Dropout (Chống Overfitting)

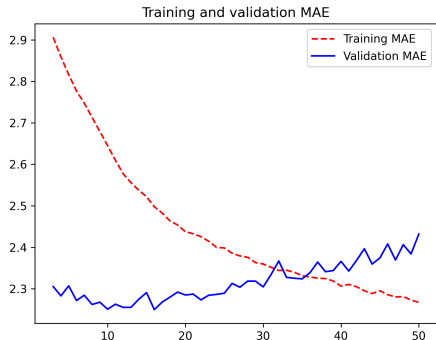
- LSTM cơ bản bị Overfit (Đường Validation và Training phân kì nhanh).
- Sử dụng `recurrent_dropout` để tắt ngẫu nhiên một số kết nối "bên trong" tế bào LSTM thay vì chỉ tắt dữ liệu đầu vào.
- **Kết quả:** Mô hình duy trì độ ổn định tuyệt vời, Validation Loss giảm sâu hơn nữa và không bị dội ngược (Overfit).



Hình 13.10: Sức mạnh của Recurrent Dropout

Kỹ thuật 2: Xếp chồng RNN (Stacked GRU)

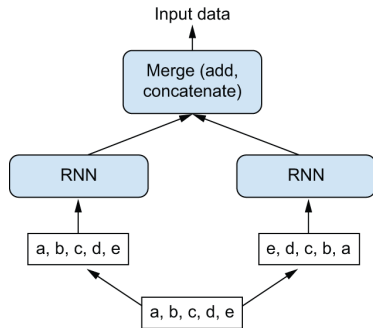
- Khi mô hình không còn Overfit, làm sao để tăng hiệu năng tiếp? → Tăng độ phức tạp.
- Xếp chồng nhiều lớp RNN lên nhau (Stacked RNN).
- **Lưu ý:** Các lớp bên dưới phải thiết lập `return_sequences=True` để trả về toàn bộ chuỗi t thay vì chỉ kết quả cuối cùng.
- GRU (Gated Recurrent Unit) là một phiên bản rút gọn, nhẹ hơn và chạy nhanh hơn LSTM.



Hình 13.11: Xếp chồng GRU + Dropout

Kỹ thuật 3: Mạng truyền hồi Hai chiều (Bidirectional RNN)

- RNN thông thường chỉ đọc dữ liệu theo chiều thời gian (Từ quá khứ tới hiện tại).
- Bidirectional RNN dùng 2 lớp RNN độc lập: 1 đọc xuôi, 1 đọc ngược (từ tương lai về quá khứ), sau đó gộp kết quả lại.
- Vô cùng hữu ích trong xử lý ngôn ngữ tự nhiên (NLP) khi ý nghĩa của một từ phụ thuộc vào những từ phía sau nó.



Hình 13.12: Kiến trúc Bidirectional RNN

- **Timeseries (Chuỗi thời gian)** khác biệt hoàn toàn với dữ liệu thông thường vì thứ tự cấu thành nên ý nghĩa. Dense và Conv1D không phù hợp.
- **Mạng truyền hồi (RNN)** duy trì "Trạng thái" bộ nhớ, nhưng vướng phải lỗi Triệt tiêu đạo hàm (Vanishing Gradients).
- **LSTM / GRU** giải quyết triệt để lỗi trên bằng hệ thống Cổng (Gates) và Trạng thái Tế bào (Cell State C_t).
- Các kỹ thuật nâng cao như **Recurrent Dropout**, **Stacked RNN**, và **Bidirectional RNN** giúp tối ưu hóa hiệu suất tối đa.